Krishi Bazar: AI-Powered Bengali Vegetable Recognition, Market Price Tracking, and Agricultural Decision Support System

Submitted by

Md. Mahmudur Rahman

Student ID: IT24024

Session: 2023–24

2nd Year, 2nd Semester

Supervised by

Dr. Nazrul Islam

Associate Professor

Department of Information and Communication Technology

Submitted in partial fulfillment of the requirements for the degree of
Bachelor of Science (Engineering) in Information and Communication Technology



Department of Information and Communication Technology

Mawlana Bhashani Science and Technology University

Santosh, Tangail-1902, Bangladesh

November 2026

Approval

This is to certify that the project report titled **"Krishi Bazar: AI-Powered Bengali Vegetable Recognition, Market Price Tracking, and Agricultural Decesion Support System",** submitted by **Md Mahmudur Rahman** (Student ID: **IT24024**, Session: **2023–24**) from the Department of Information and Communication Technology, Mawlana Bhashani Science and Technology University, has been accepted as satisfactory in partial fulfillment of the requirements for the degree of Bachelor of Science in Information and Communication Technology.

Project Supervisor

_____

Dr. Nazrul Islam
Associate Professor,
Department of ICT,
Mawlana Bhashani Science and Technology University
Date: _____

Declaration

I declare that this report describes my project, Krishi Bazar, developed under the supervision of **Dr. Nazrul Islam**. Sources used to explain the technical background and design decisions are acknowledged through numbered citations and a reference list.

The report distinguishes the interface demonstrated in the supplied application screen-shots from proposed improvements and measurements that still require testing. No recognition accuracy, inference latency, financial benefit, or field-study outcome is claimed without supporting evaluation records.

_____

Md. Mahmudur Rahman
Student ID: IT24024
Session: 2023–24
Date: _____

Acknowledgements

All praises are due to the Almighty Allah for bestowing the strength, intellect, and perseverance required to bring this capstone project to successful completion.

I express my deepest gratitude to Mawlana Bhashani Science and Technology University (MBSTU) and the Department of Information and Communication Technology for providing the rigorous academic setting and laboratory infrastructure in which this project was conceived and developed.

I am particularly grateful to my project supervisor, **Dr. Nazrul Islam**, Associate Professor, Department of ICT, for his patient guidance, continuous technical critique, and encouragement throughout all phases of software development, AI model integration, and report preparation.

I also acknowledge the invaluable contributions of the faculty members of the Department of ICT who enriched my theoretical foundation in computer vision, mobile software engineering, and distributed cloud computing. Special thanks are extended to my peers, lab colleagues, and friends for their constructive feedback during system usability trials. Finally, I thank my family for their unconditional prayers, sacrifices, and moral support throughout my academic endeavor.

Md Mahmudur Rahman

Abstract

Krishi Bazar is a Bengali-first application that brings vegetable market information and an AI image-recognition workflow into one interface. The project uses Flutter and Dart for the client, Cloud Firestore for market records, Firebase Authentication for administrator sign-in, and the Gemini API for image analysis. The supplied interface demonstrates searchable market listings, district filters, wholesale and retail prices, camera or gallery input, vegetable-based price comparison, and an administrative price-entry form. The design also supports presenting vegetable information in Bengali. This report explains the architecture, data model, technologies, security considerations, and evaluation plan. Application screenshots document interface implementation, but they do not establish recognition accuracy, response time, nationwide data coverage, or production readiness. Future work includes a labelled recognition dataset, controlled end-to-end tests, protected AI credentials, and verified market-data integration.

Keywords: Bengali interface, vegetable recognition, market prices, Flutter, Firebase, Gemini API.

Contents

Approval i

Declaration ii

Acknowledgements iii

Abstract iv

Table of Contents v

List of Figures vii

List of Tables viii

1 Introduction 1

1.1 Background and Motivation 1

1.2 Problem Statement 1

1.3 Purpose and Objectives 2

1.4 Scope and Contributions 2

1.5 Organization of the Report 3

2 Background and Related Work 4

2.1 Agricultural Image Analysis 4

2.2 Multimodal Generative Models 4

2.3 Market Information Services 5

2.4 Mobile and Cloud Technologies 5

2.5 Comparison and Research Gap 5

3 Methodology 6

3.1 Development Approach and Evidence 6

3.2 System Architecture 6
3.3 AI Recognition Workflow 7
3.4 Market Data Model and Comparison Logic 7
3.5 Technologies and Development Tools 8
3.6 Application Features 9
3.7 Security and Data Handling 10

4 Investigation, Results, Analysis and Discussion 11
4.1 Evidence and Evaluation Boundaries 11
4.2 Application Interface Results 11
4.3 Proposed Test Plan 16
4.4 Recognition and Performance Evaluation 17
4.5 Discussion 18

5 Impacts and Design Constraints 19
5.1 Societal and Cultural Considerations 19
5.2 Information Quality and Safety 19
5.3 Environmental and Economic Considerations 19
5.4 Technical Constraints 19
5.5 Ethics and Professional Responsibility 20

6 Project Planning and Budget 21
6.1 Work Plan 21
6.2 Budget and Financial Feasibility 21

7 Complex Engineering Problems and Activities 23
7.1 Engineering Problem Analysis 23
7.2 Engineering Activities 23
7.3 Trade-offs 24

8 Conclusion 25

References 26

List of Figures

3.1 Logical system architecture of the prototype 6
3.2 AI request and response-validation workflow 7
3.3 Technologies and development-tool artwork supplied for the project . 8

4.1 Home and market-price listing..... 12
4.2 AI scan interface with camera and gallery input 13
4.3 Vegetable and district-based price comparison..... 14
4.4 Application information and feature overview..... 15
4.5 Administrative price-entry interface, with sensitive fields redacted 16

List of Tables

2.1 Related approaches and their relevance to Krishi Bazar 5

3.1 Logical market-price record and validation requirements 8

3.2 Technology stack and its role 9

3.3 Features and the available evidence 9

4.1 Functional tests to execute before deployment 17

6.1 Proposed completion and verification schedule 21

6.2 Budget categories and estimation basis 22

7.1 Engineering challenges and required analysis 23

Chapter 1 Introduction

1.1 Background and Motivation

Vegetable buyers and sellers need market information that identifies the commodity, market, unit, price type, and reporting date. Bangladesh's Department of Agricultural Marketing (DAM) publishes a daily price-report portal, providing an official reference for agricultural price information [1]. The existence of such a service motivates applications that present prices in a convenient local-language interface. It does not establish that any particular app already receives an official data feed.

Image analysis offers another way to access vegetable information when a user does not know the product's name. Mohanty, Hughes, and Salathé examined image-based plant disease classification, illustrating the potential of visual agricultural applications and the importance of evaluation beyond controlled images [2]. That study concerns plant disease, not Krishi Bazar's vegetable identification task, so its results cannot be transferred to this project.

Krishi Bazar brings market browsing and image-based queries into one Bengali interface. Flutter supplies the client-development framework [3], while Gemini supports image-and-text input [4]. These technologies make the proposed integration feasible, but reliable operation still depends on data quality, network access, validation, and secure deployment.

1.2 Problem Statement

A price displayed without its unit, date, market, or source can mislead a user. Similarly, an AI-generated vegetable label can be incorrect even if its explanation sounds convincing. The engineering problem is therefore to connect a clear Bengali interface with traceable market records and an image-analysis workflow that handles uncertainty.

The application addresses three practical tasks: finding price records, comparing comparable records, and requesting vegetable information from an image. It must distin-

guish demonstration data from verified market updates. It must also prevent ordinary users from modifying administrative records. Firebase documentation separates user authentication from authorization through security rules, which informs this requirement [5, 6].

1.3 Purpose and Objectives

The project's goal is to develop a usable prototype for Bengali-speaking users. Its objectives are to:

1. Present searchable vegetable prices with market and district filters.
2. Show wholesale and retail prices with explicit units and update dates.
3. Provide camera and gallery entry points for AI-assisted identification.
4. Compare prices for the same vegetable using a consistent price type and unit.
5. Provide an administrator interface for market-price entry and maintenance.
6. Document limitations and define a reproducible evaluation procedure.

1.4 Scope and Contributions

The supplied screenshots demonstrate home, scan, comparison, information, and administrative interfaces. The contribution is the integration of these workflows in a Bengali application rather than a new machine-learning architecture. The system calls a pretrained AI service. It does not train MobileNet, ResNet, or a custom vision model.

Official DAM integration, forecasting, voice assistance, verified nationwide coverage, and fully tested offline operation remain outside the demonstrated scope. The report does not claim that displaying a price difference proves an intermediary's profit or establishes unfair trading.

1.5 Organization of the Report

Chapter 2 reviews relevant research and technologies. Chapter 3 explains the methodology, architecture, data model, and feature design. Chapter 4 presents the application interfaces and discusses evaluation. Chapters 5 and 6 address impacts, constraints, planning, and costs. Chapter 7 discusses engineering complexity. Chapter 8 concludes the report.

Chapter 2

Background and Related Work

2.1 Agricultural Image Analysis

Mohanty et al. used deep learning for plant disease classification and showed that a carefully defined image task requires an appropriate dataset and evaluation protocol [2]. For Krishi Bazar, a useful lesson is to separate the model's potential from the app's measured performance. Photographing a vegetable against a cluttered background presents different conditions from classifying a curated leaf image.

Sandler et al. introduced MobileNetV2, an efficient neural-network design for mobile vision [7]. He et al. introduced residual learning to improve the optimization of deep networks [8]. These are relevant alternatives for a future locally executed classifier. Neither architecture is presented as part of the current application. Adopting one would require training data, class definitions, deployment work, and a separate accuracy assessment.

2.2 Multimodal Generative Models

The Gemini 1.5 technical report describes multimodal processing of different input types [9]. Current Gemini image-understanding documentation explains image input to API requests [4]. Together, these sources provide background for an image-and-text recognition workflow. The report uses "Gemini API" for the project integration because the exact deployed model identifier and package lockfile were not supplied.

Structured output can constrain the format of a response [10]. It does not verify the vegetable species, establish nutritional truth, or make a generated confidence value statistically calibrated. The app therefore needs field validation and an uncertainty state even when the response is syntactically valid JSON.

2.3 Market Information Services

DAM's daily price-report service is a relevant existing information source [1]. Its role differs from a visual recognition model: it reports market prices rather than recognizing objects in photographs. Krishi Bazar aims to place these user tasks near one another. The screenshots do not establish a direct connection to DAM, and no automated import is claimed.

2.4 Mobile and Cloud Technologies

Flutter offers a widget-based framework, and Dart supplies the programming language [3, 11]. Flutter's architecture guide recommends separating presentation and data responsibilities [12]. Cloud Firestore supports listeners for database changes [13]; receiving a database update, however, does not prove that a price was recently collected or independently verified.

Firestore also provides offline access to cached data on supported platforms, subject to configuration and platform behavior [14]. This capability should be tested in the actual app. It is different from shipping static demo records and does not prove that a separate SQLite cache has been implemented.

2.5 Comparison and Research Gap

Table 2.1: Related approaches and their relevance to Krishi Bazar

Approach	Relevant capability	Boundary for this project
DAM price portal [1]	Official agricultural price reports	Not an image-recognition service; no app integration demonstrated
Plant image research [2]	Agricultural image classification	Different task and dataset; accuracy is not transferable
MobileNetV2 [7]	Efficient mobile visual model	Possible future alternative, not the current AI backend
Gemini API [4, 10]	Image input and structured responses	Needs validation, network access, and secure access
Krishi Bazar prototype	Bengali market, scan, comparison, and admin interfaces	End-to-end accuracy and usability remain unmeasured

The project focuses on integration and local-language presentation. It makes no claim that other applications universally lack these features. Its next research task is to test whether the combined workflow improves task completion for its intended users.

Chapter 3 Methodology

3.1 Development Approach and Evidence

The report follows a prototype-oriented approach: define user tasks, design the interfaces, connect services, inspect outputs, and plan controlled tests. Flutter's separation-of-concerns guidance informs the component boundaries [12]. The supplied LaTeX draft and screenshots document the intended design and visible interface. The application repository, dependency lockfile, test logs, and backend rules were not provided, so exact implementation details require source-code verification.

3.2 System Architecture

Figure 3.1 shows the logical integration of the client and services. The client presents market records and receives image input. A market service accesses Firestore, an authentication service manages sign-in, and an AI service requests image analysis. A protected server-side AI gateway is a recommended deployment improvement rather than a demonstrated component [6, 15].

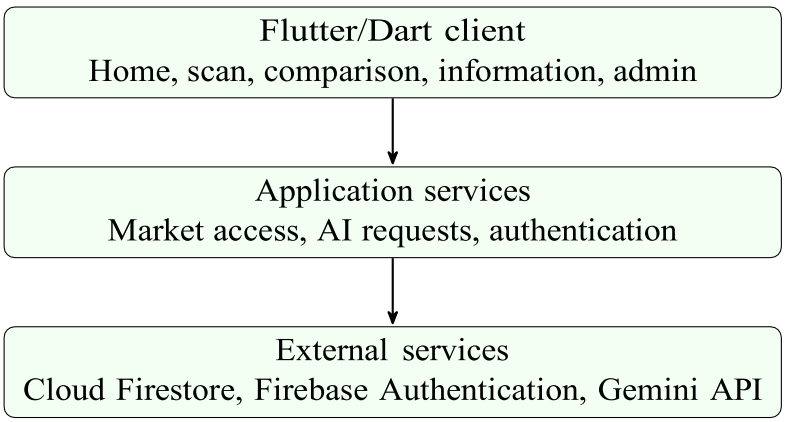


Figure 3.1: Logical system architecture of the prototype

This diagram describes responsibilities rather than asserting a verified implementation of a named architectural pattern. In particular, the supplied evidence is insufficient

to establish that a repository layer or custom local cache exists.

3.3 AI Recognition Workflow

Gemini image requests combine an image with task instructions [4]. The proposed processing sequence appears in Figure 3.2.

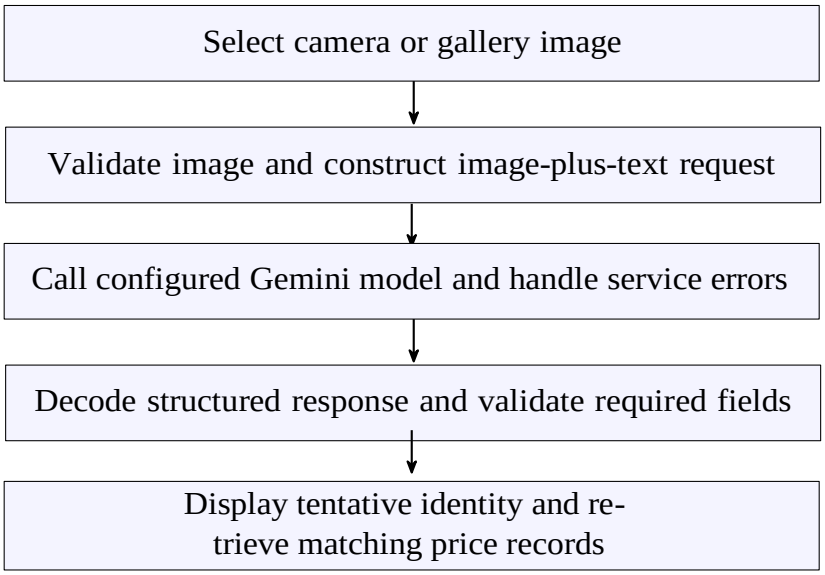


Figure 3.2: AI request and response-validation workflow

The response should include only fields needed by the interface, such as a vegetable label, scientific-name suggestion, short description, and relevant warnings. Generated nutritional or cultivation information requires independent verification before users rely on it. Structured output support can improve format consistency [10], but a prompt or regular expression alone is not a complete schema validator.

Timeouts, invalid JSON, unsupported files, unavailable models, and uncertain identifications should produce clear error states. A fallback model must follow the same validation contract. The draft's image dimensions, compression quality, retry count, model version, and temperature settings are not treated as verified configuration without source-code evidence.

3.4 Market Data Model and Comparison Logic

Table 3.1 provides a recommended logical record model derived from visible price-card fields. Exact collection and property names must match the final application code. Firestore stores documents and supports real-time listeners [13].

Table 3.1: Logical market-price record and validation requirements

Field	Type	Purpose / validation
Record identifier	String	Unique record or document identity
Vegetable name	String	Required commodity label
Market name	String	Required local market label
District	String	Filter and location context
Wholesale price	Number	Nonnegative value for stated unit
Retail price	Number	Nonnegative value for stated unit
Unit	String	Comparable unit, such as kilogram
Availability	Enum	Defined availability category
Updated time	Timestamp	Date/time of the recorded update
Source label	String	Demo, admin entry, or verified external source
Verification state	Enum	Proposed distinction between unverified and checked data

For comparable records with the same commodity, unit, and price type, the displayed spread may be computed as

$$\Delta p = \max_{i \in V} p_i - \min_{i \in V} p_i, \tag{3.1}$$

where V contains the valid records after filtering. If $|V| = 1$, the spread equals zero. If V is empty, the interface should show an unavailable-data message. A spread is a difference among quoted prices, not a measure of net trader profit. Transport, quality, time, losses, and other costs require separate data.

3.5 Technologies and Development Tools

Figure 3.3 includes the technology artwork supplied with the project. Table 3.2 explains the stack without asserting unverified dependency versions.



Flutter Firebase Visual Studio Code

Figure 3.3: Technologies and development-tool artwork supplied for the project

Table 3.2: Technology stack and its role

Technology	Project role and reference
Flutter	Client UI and navigation [3]
Dart	Application logic, asynchronous operations, and data handling [11]
Material components	Navigation, controls, and interface styling [16]
Gemini API	Image-analysis service integration [4]
Cloud Firestore	Market-record storage and update delivery [13]
Firebase Authentication	Administrator sign-in [5]
Firestore Security Rules	Required authorization and data validation [6]
Visual Studio Code	Flutter-capable editor and debugging environment [17] User-
Camera/gallery input	facing image acquisition, visible in the supplied UI

Exact SDK, plugin, IDE, and model versions should be recorded from the project environment and dependency lockfile before reproducible testing. The documentation describes platform capabilities; it is not evidence that every supported platform has been built or tested for this app.

3.6 Application Features

Table 3.3: Features and the available evidence Feature

	Interface support	Evidence boundary
Bengali navigation	Home, scan, comparison, about	Visible in screenshots
Market search/filter	Search box and district chips	Query correctness needs testing
Price information	Wholesale, retail, unit, update label	Data provenance needs checking
Image input	Camera and gallery controls	Recognition result not supplied
Price comparison	Vegetable/district selection, price toggle	Multi-market correctness needs testing
Admin entry	Vegetable, market, district, price inputs	Save and permission tests not supplied
Vegetable details	About screen advertises scientific name, nutrition, seasons	Detail output needs verification
Demo seeding	Admin screen includes seed control	Demo is not an official market feed

3.7 Security and Data Handling

An administrator sign-in screen cannot by itself enforce write authorization. Firestore rules should validate the user's role and reject malformed or unauthorized writes [5,6]. Gemini access credentials should be protected according to provider guidance [15]. The supplied admin screenshot contains account information and a credential-like value; both are redacted in the report assets.

A production design should use a protected service boundary for privileged AI requests, limit request volume, and avoid exposing secrets through client bundles or readable database documents. Images should be transmitted only for the stated purpose, with users informed about external AI processing and any retention policy.

Chapter 4

Investigation, Results, Analysis and Discussion

4.1 Evidence and Evaluation Boundaries

The results in this chapter are interface observations from the supplied screenshots. They establish visible controls and displayed content. They do not establish successful backend writes, secure rules, recognition accuracy, cross-device compatibility, or response-time performance. The report therefore separates demonstrated interface features from a proposed testing plan.

4.2 Application Interface Results

4.2.1 Home and Market Listing

Figure 4.1 shows Bengali navigation, a search field, district filters, and market-price cards. The visible eggplant card lists wholesale and retail prices of BDT 40 and BDT 60 per kilogram. These are values shown in the project screenshot, not independently verified current market quotations. The interface includes a source label and update date, supporting contextual price presentation.

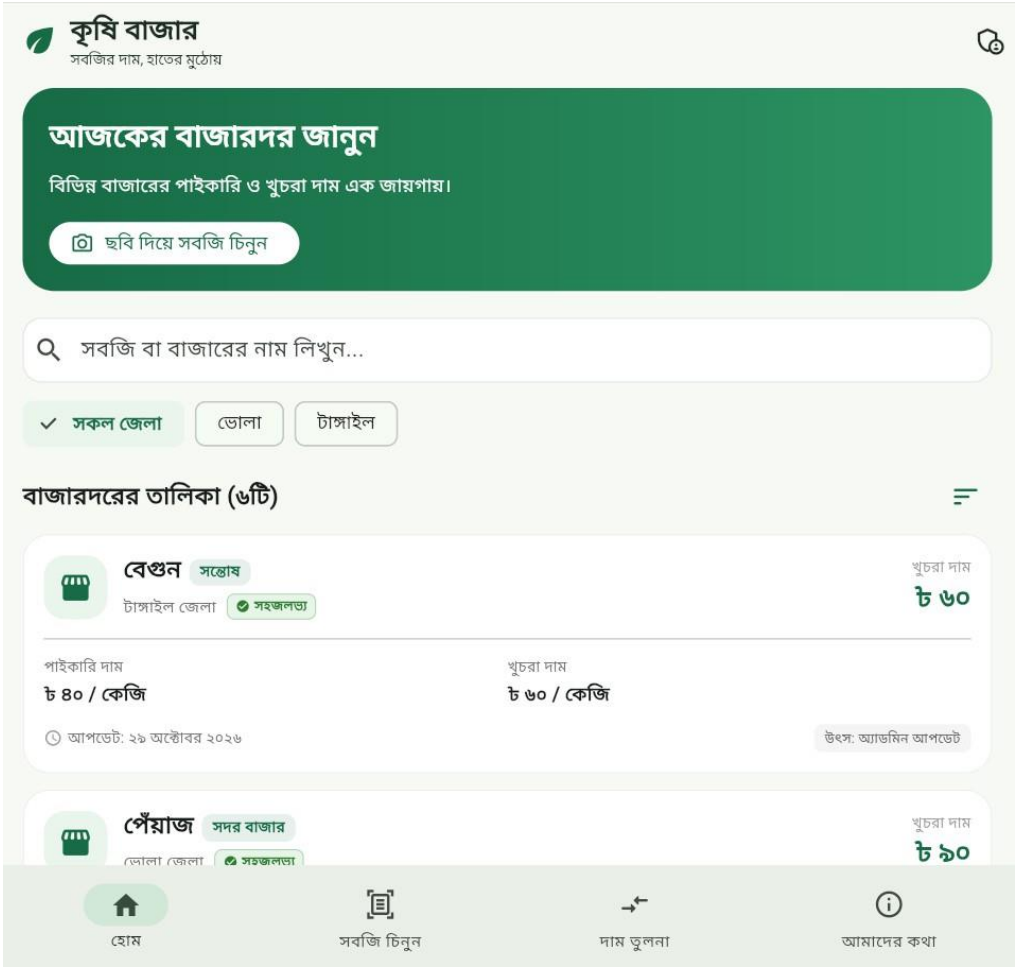


Figure 4.1: Krishi Bazar home interface showing district filters, search, and whole-sale/retail price cards. Source: supplied application screenshot.

4.2.2 Camera and Gallery Input

Figure 4.2 shows an image-preview area, camera and gallery controls, and a short explanation of the intended AI feature. The preview is empty. Accordingly, this figure documents the input interface rather than a successful vegetable prediction or measured AI result.

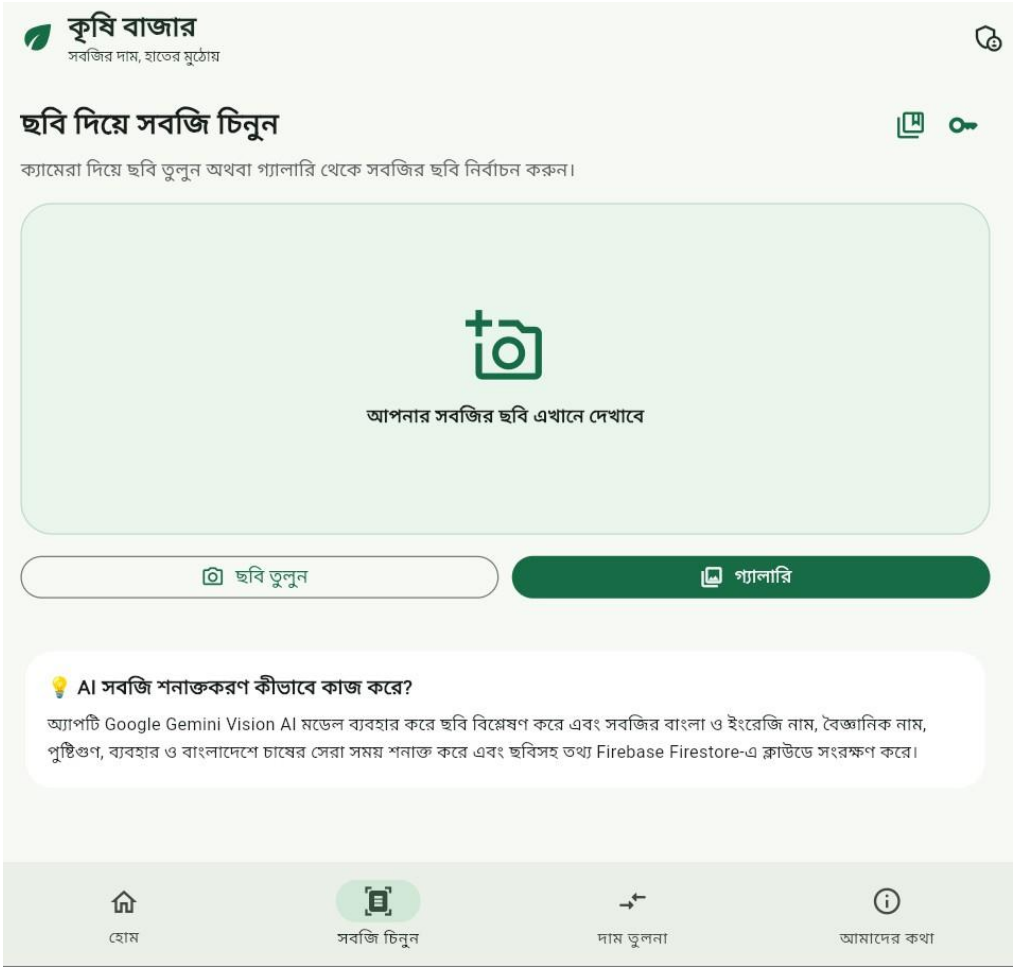


Figure 4.2: Vegetable image-input screen. The empty preview demonstrates the scan interface, not an evaluated recognition output. Source: supplied application screenshot.

4.2.3 Market-Price Comparison

Figure 4.3 shows a potato selection, a district selector, retail/wholesale controls, and one displayed market record. The retail value shown is BDT 35 per kilogram. With one comparable record, the displayed maximum price difference is zero. This is consistent with the spread formula in Chapter 3, but additional records and controlled tests are needed to verify comparison behavior generally.

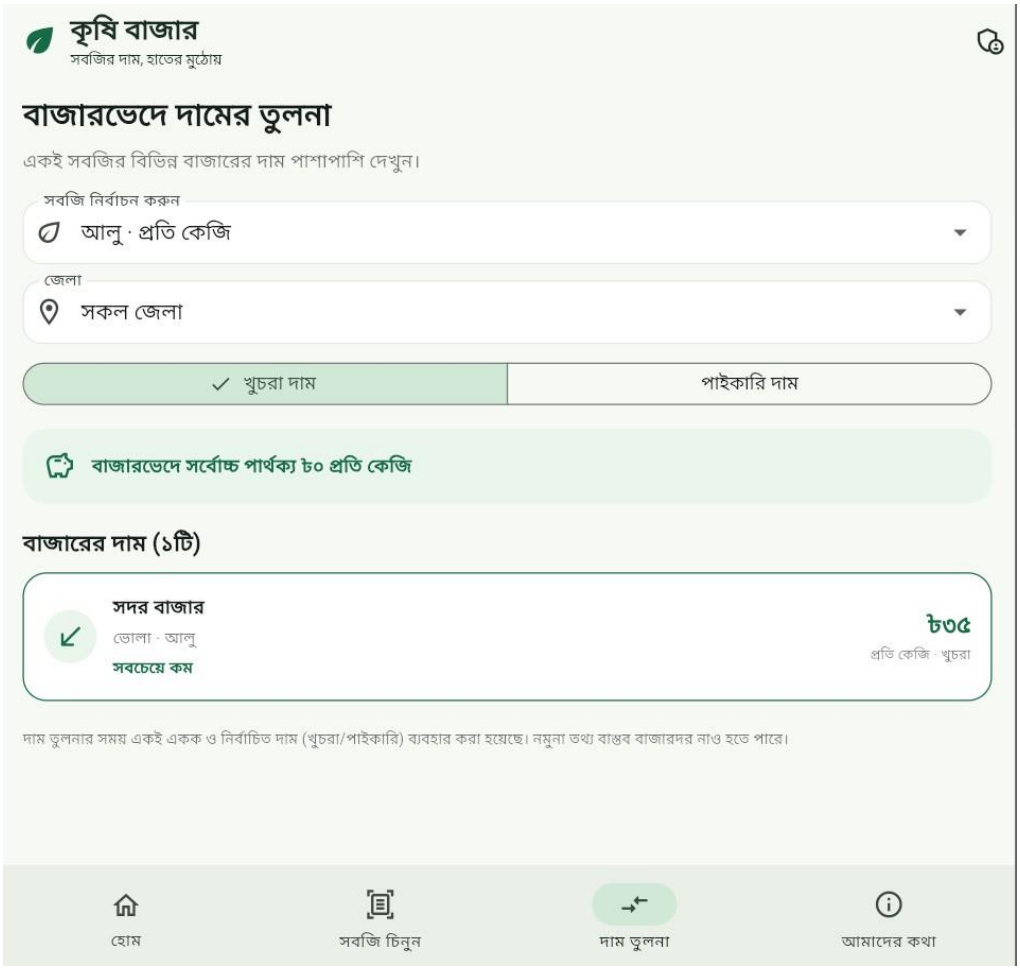


Figure 4.3: Price-comparison interface showing one potato record and a zero spread for the displayed selection. Source: supplied application screenshot.

4.2.4 Information and Feature Overview

Figure 4.4 presents the application's stated purpose and feature list. It advertises image recognition, scientific names, nutritional information, cultivation seasons, Firestore-backed prices, and market comparison. Its notice distinguishes demonstration prices from guaranteed real-world quotations. These advertised features guide the evaluation requirements; the screenshot alone does not verify each one.



Figure 4.4: About screen describing the app and its intended features. Source: supplied application screenshot.

4.2.5 Administrative Interface

Figure 4.5 shows demonstration-data seeding, AI configuration, and fields for adding a market-price record. The report preserves the interface while redacting account information and the credential field. The presence of a save control does not establish that unauthorized writes are blocked or that submitted values persist correctly.

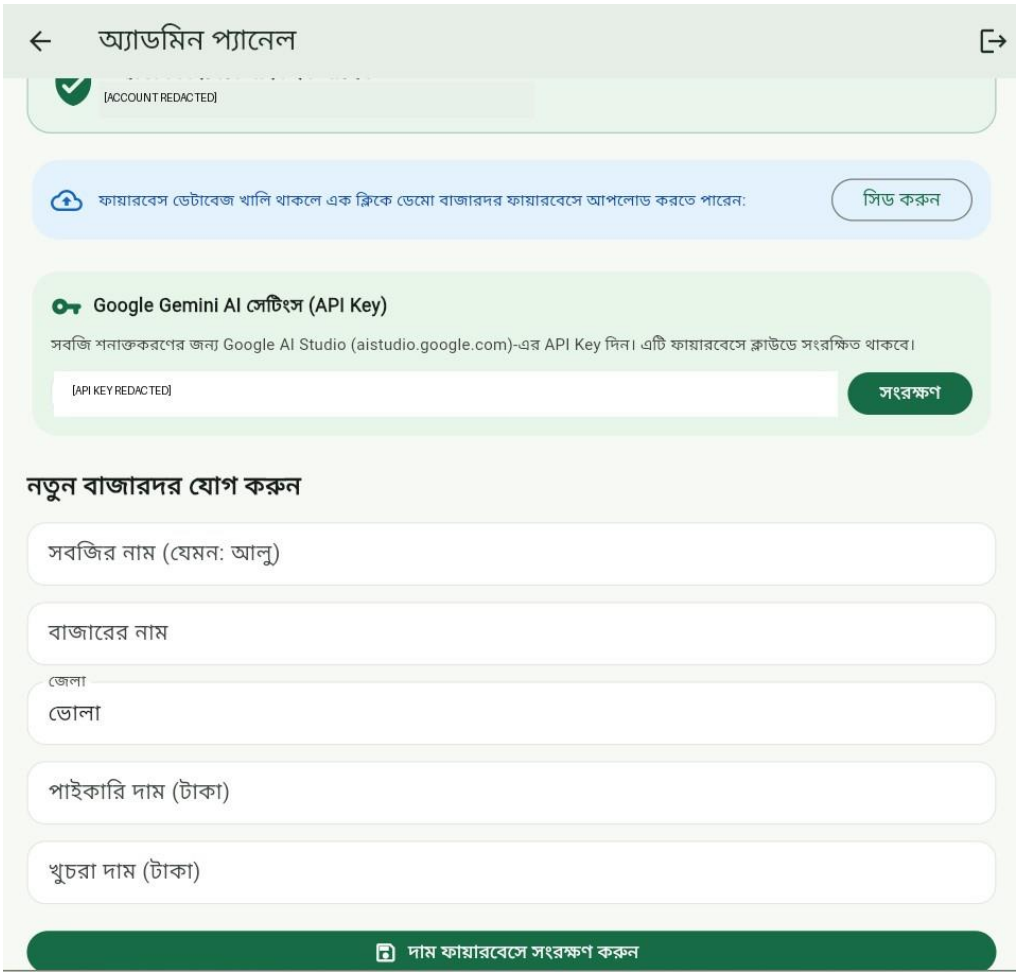


Figure 4.5: Administrator panel for configuration, demo seeding, and price entry. Ac-count information and the credential-like value are redacted. Source: supplied application screenshot, redacted for this report.

4.3 Proposed Test Plan

Flutter distinguishes unit, widget, and integration testing [18]. The following plan applies those levels to this project's risks. It is a plan, not a record of passed tests.

Table 4.1: Functional tests to execute before deployment Area

	Test input or action	Expected result
Search/filter	Known name, missing name, changed district	Correct matching records and explicit empty state
Comparison	Zero, one, and several same-unit records	Correct spread and consistent selected price type
Price validation	Negative, blank, nonnumeric, and extreme values	Invalid writes rejected with feedback
AI parsing	Valid, malformed, incomplete, uncertain response	Safe handling without fabricated details
Authentication	Signed-out, normal user, authorized admin	Privileged writes allowed only to authorized role
Network	Disconnect, reconnect, timeout	Explicit state and safe retry behavior
Demo records	Empty live data and seeded data	Clear source labels, no claim of live official data

4.4 Recognition and Performance Evaluation

A future dataset should contain independently labelled vegetable images from the target use context. Include lighting changes, backgrounds, damaged produce, visually similar varieties, and non-vegetable inputs. Separate prompt development from evaluation, and retain model identifiers, dates, and request configuration.

For N independently labelled test images, top-1 accuracy can be computed as

$$\text{Accuracy} = \frac{N_{\text{correct}}}{N} \times 100\%. \tag{4.1}$$

Precision and recall require class-wise counts and should not be used as interchangeable names for accuracy. Record rejected or uncertain cases separately. Measure end-to-end time from user submission to the completed display, reporting median and tail latency together with network conditions and failed-request counts.

No dataset or timing log accompanies the screenshots. The earlier draft's 96.2% accuracy, 1.42-second latency, 300 trials, and millisecond cache timings are therefore excluded from the reported results. Firestore caching capability [14] does not supply a measured performance result for this application.

4.5 Discussion

The interface supports the intended task structure and makes market context visible. The principal remaining questions concern correctness and trust: whether AI outputs match labelled images, whether prices retain their provenance, whether comparisons normalize units, and whether administrative access is enforced. Addressing these questions is more meaningful than adding unsupported benchmark numbers.

Chapter 5

Impacts and Design Constraints

5.1 Societal and Cultural Considerations

A Bengali interface may make the app easier to approach for users who prefer Bengali, but accessibility and usability require user testing. WCAG 2.2 provides guidance relevant to readable content, contrast, focus visibility, and interaction [19]. Future evaluation should include Bengali speakers with different levels of technical experience and should measure task completion rather than assuming universal ease of use.

5.2 Information Quality and Safety

Incorrect prices can influence buying or selling decisions. Every record should show its market, unit, timestamp, and source. AI-generated species, nutrition, and cultivation statements should be identified as tentative unless checked against authoritative references. The application should not encourage users to treat an uncertain visual prediction as a guarantee of food identity or suitability.

5.3 Environmental and Economic Considerations

Improved information access could support better purchasing and distribution decisions, but no reduction in waste, travel, emissions, or household costs has been measured. Cloud requests also consume network and computing resources. Image sizes, retries, and retention should therefore be managed according to actual need.

5.4 Technical Constraints

Cloud image analysis depends on service availability and network connectivity. Data freshness depends on reporting practices, not just database synchronization. Device display size, Bengali text scaling, long commodity names, and varying input quality

affect usability. Firestore cache behavior must be evaluated on each target platform [14].

5.5 Ethics and Professional Responsibility

Users should know when an image goes to an external AI service. The project should collect only necessary information, control access, and avoid publishing credentials [6, 15]. Model output can remain incorrect despite low temperature or strict formatting. Ethical reporting therefore includes visible uncertainty, honest evaluation boundaries, and a route for correcting inaccurate market entries.

Chapter 6

Project Planning and Budget

6.1 Work Plan

Table 6.1 is a proposed completion plan. It does not assert that field interviews, usability trials, or a 16-week implementation schedule have already occurred.

Table 6.1: Proposed completion and verification schedule

Phase	Duration	Deliverable
Requirements review	Week 1	Confirm scope and user tasks
Source and configuration review	Week 2	Dependency inventory, schema, model settings
Service hardening	Weeks 3–4	Protected AI access and validated database rules
Functional testing	Weeks 5–6	Unit, widget, and integration test records
Recognition evaluation	Weeks 7–8	Labelled dataset and reproducible results
User review and documentation	Weeks 9–10	Usability findings and corrected final report

6.2 Budget and Financial Feasibility

No invoices, usage logs, or developer-hour records were supplied. The budget is therefore a planning framework rather than a verified zero-cost project total. Firebase and Gemini have pricing and usage conditions that should be checked for the selected services and models [20, 21].

Table 6.2: Budget categories and estimation basis

Item	Estimation basis	Current status
Development tools	Framework/editor licensing and existing hardware	No new hardware purchase evidenced
Gemini requests	Model, image input, output, and request volume	Usage-based estimate required
Firestore	Reads, writes, storage, and relevant transfer costs	Usage logs not supplied
Authentication	Provider and usage conditions	Configuration must be reviewed
Hosting/gateway	Deployment and server-side AI access	Production design pending
Testing/data collection	Images, annotation, devices, and participant time	Planned work

For planning, the operating budget can be represented as

$$C_{\text{total}} = C_{\text{AI}} + C_{\text{database}} + C_{\text{hosting}} + C_{\text{network}} + C_{\text{testing}}. \tag{6.1}$$

Each term needs an explicit workload and pricing date. Free-tier eligibility does not guarantee that a public deployment will remain free.

Chapter 7

Complex Engineering Problems and Activities

7.1 Engineering Problem Analysis

This chapter provides a qualitative discussion of complexity. It does not claim formal accreditation compliance or that institutional P1–P7 criteria have been independently assessed. Any required institutional mapping should be checked against the department's current rubric.

Table 7.1: Engineering challenges and required analysis

Challenge	Analysis and verification needed
Multimodal uncer-	Distinguish valid JSON from correct identification; evaluate tainty uncertain cases
Price comparability	Align vegetable identity, unit, reporting time, district, and price type
Trust boundaries	Separate public reads, administrator writes, and privileged AI credentials
Service dependence	Define timeout, retry, unavailable-service, and empty-data behavior
Bengali presenta-	Check long text, numeral display, input, and accessibility tion
Evidence quality	Separate observed interface behavior from proposed and measured outcomes

7.2 Engineering Activities

The project combines interface design, cloud-data integration, image-service orchestration, and technical documentation. The activities interact: an AI label must map to a meaningful price query, an administrator's update must retain provenance, and a failed request must produce understandable feedback.

Separation of responsibilities helps these activities remain testable [12]. Security rules govern database access [6], while structured AI responses reduce format ambigu-

ity [10]. These tools support engineering control but do not replace tests or a threat review.

7.3 Trade-offs

A cloud AI service avoids packaging and training a local classifier but introduces connectivity, service-cost, and credential-management requirements. A future MobileNetV2-based alternative could move inference onto the device [7], but would require a task-specific dataset and deployment evaluation. Similarly, permissive database access speeds demonstrations but is unsuitable for public administration. The final design should prioritize explicit data quality and authorization over convenience alone.

Chapter 8

Conclusion

Krishi Bazar presents a Bengali-first interface for vegetable market information and an AI-assisted identification workflow. The project combines Flutter and Dart with Firebase services and the Gemini API, linking market browsing, image input, price comparison, and administrative data entry within one application. The supplied screenshots document these interfaces and show how users can inspect wholesale and retail prices, select districts, and access camera or gallery controls.

The present evidence supports interface implementation rather than measured recognition accuracy, response speed, nationwide coverage, or verified economic benefits. Reliable deployment therefore requires controlled functional tests, a labelled vegetable-image dataset, and checks of data provenance, units, timestamps, and administrative authorization. Generated descriptions should communicate uncertainty and should not replace expert agricultural or nutritional guidance.

Future development should protect AI credentials behind a secure service boundary, validate Firestore rules, distinguish demonstration records from verified updates, and assess usability with Bengali-speaking participants. Integration with an authoritative market-data source and reproducible end-to-end evaluation would strengthen the application and provide a sound basis for expanding its features.

References

[1] Department of Agricultural Marketing, Government of Bangladesh, “Daily Price List Report.” [Online]. Available: https://market.dam.gov.bd/market_daily_price_report?L=E. Accessed: Nov. 2, 2026.

[2] S. P. Mohanty, D. P. Hughes, and M. Salathé, “Using deep learning for image-based plant disease detection,” *Frontiers in Plant Science*, vol. 7, art. 1419, 2016. doi: 10.3389/fpls.2016.01419.

[3] Flutter, “Flutter documentation.” [Online]. Available: https://docs.flutter.dev/. Accessed: Nov. 2, 2026.

[4] Google, “Image understanding,” *Gemini API documentation*. [Online]. Available: https://ai.google.dev/gemini-api/docs/image-understanding. Accessed: Nov. 2, 2026.

[5] Google Firebase, “Get started with Firebase Authentication on Flutter.” [Online]. Available: https://firebase.google.com/docs/auth/flutter/start. Accessed: Nov. 2, 2026.

[6] Google Firebase, “Writing conditions for Cloud Firestore Security Rules.” [Online]. Available: https://firebase.google.com/docs/firestore/security/ rules-conditions. Accessed: Nov. 2, 2026.

[7] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *Proc. IEEE/CVF CVPR*, 2018, pp. 4510–4520. https://openaccess.thecvf.com/content_cvpr_2018/html/Sandler_MobileNetV2_Inverted_Residuals_CVPR_2018_paper.html.

[8] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE CVPR*, 2016, pp. 770–778. Preprint: https://arxiv.org/abs/1512.03385.

[9] Gemini Team, “Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context,” arXiv:2403.05530, 2024. https://arxiv.org/abs/2403.05530.

[10] Google, “Structured outputs,” *Gemini API documentation*. [Online]. Avail-able: https://ai.google.dev/gemini-api/docs/structured-output. Ac-cessed: Nov. 2, 2026.

[11] Dart, “Dart overview.” [Online]. Available: https://dart.dev/overview. Accessed: Nov. 2, 2026.

[12] Flutter, “Guide to app architecture.” [Online]. Available: https://docs.flutter.dev/app-architecture/guide. Accessed: Nov. 2, 2026.

[13] Google Firebase, “Get realtime updates with Cloud Firestore.” [Online]. Available: https://firebase.google.com/docs/firestore/query-data/listen. Accessed: Nov. 2, 2026.

[14] Google Firebase, “Access data offline.” [Online]. Available: https://firebase.google.com/docs/firestore/manage-data/enable-offline. Accessed: Nov. 2, 2026.

[15] Google, “Using Gemini API keys.” [Online]. Available: https://ai.google.dev/gemini-api/docs/api-key. Accessed: Nov. 2, 2026.

[16] Flutter, “Material component widgets.” [Online]. Available: https://docs.flutter.dev/ui/widgets/material. Accessed: Nov. 2, 2026.

[17] Flutter, “Visual Studio Code.” [Online]. Available: https://docs.flutter.dev/tools/vs-code. Accessed: Nov. 2, 2026.

[18] Flutter, “Testing.” [Online]. Available: https://docs.flutter.dev/cookbook/testing. Accessed: Nov. 2, 2026.

[19] World Wide Web Consortium, “Web Content Accessibility Guidelines (WCAG) 2.2,” W3C Recommendation, 2024. https://www.w3.org/TR/WCAG22/.

[20] Google Firebase, “Firebase pricing.” [Online]. Available: https://firebase.google.com/pricing. Accessed: Nov. 2, 2026.

[21] Google, “Gemini Developer API pricing.” [Online]. Available: https://ai.google.dev/gemini-api/docs/pricing. Accessed: Nov. 2, 2026.